

BINX TECH

BACKEND DEVELOPMENT INTERNSHIP PROGRAM (.NET)

SPRINT 3

Planning & Day 1 Implementation Record

Cardiac Patient Monitoring System

WEEK 8 • PHASE 3 • ADVANCED QUERIES, CACHING & PERFORMANCE

Prepared for BinX Tech Internship Documentation

1. Sprint 3 Overview

Sprint focus

Week 8 is Phase 3 Sprint 3. The official internship outline places this sprint on advanced queries (LINQ), caching (Redis), and performance tuning.

Field	Detail
Sprint	Sprint 3 — Week 8
Phase	Phase 3 — Applied Project Work
Project	Cardiac Patient Monitoring System
Primary focus	Advanced queries, caching, and performance tuning
Current evidence	Appointments Summary endpoint implemented and verified through Postman

2. Sprint Goal

Goal: Improve the Cardiac Patient Monitoring System's data-access performance by identifying inefficient query patterns, using LINQ appropriately, introducing caching where it provides value, and validating the effect through observable API behavior.

3. Sprint Objectives

- Review and improve advanced EF Core / LINQ query patterns used by the project.
- Identify the N+1 query problem in endpoints that load related patient and doctor data.
- Optimize data retrieval so related data is obtained efficiently rather than issuing repeated database queries.
- Introduce Redis caching for suitable read-heavy data and define an appropriate cache strategy.
- Measure API behavior and database activity before and after optimization.
- Keep the implementation aligned with the project's existing REST API and authentication structure.

4. Sprint Backlog

Task	Expected outcome	Status
Appointments summary baseline	Working GET /api/appointments/summary endpoint with readable summary DTO	Completed — Day 1
N+1 query identification	Recognize repeated patient/doctor SELECT statements in the current implementation	Completed — Day 1
Query optimization	Replace inefficient related-data access with a more efficient EF Core/LINQ approach	Planned
Redis caching	Cache suitable read-heavy responses/data with clear invalidation considerations	Planned
Performance evidence	Compare observable request/database behavior before and after optimization	Planned

5. Sprint Day-by-Day Plan

Day	Focus	Planned work
Day 1	Baseline & N+1 analysis	Implement/verify an appointments summary query, inspect generated SQL, and establish the performance baseline.
Day 2	LINQ / EF Core optimization	Refactor inefficient related-data access and validate the optimized query behavior.
Day 3	Redis caching	Introduce caching for an appropriate read-heavy operation and verify cache behavior.
Day 4	Performance tuning	Measure and refine query/caching behavior; document performance evidence and trade-offs.
Day 5	Review & sprint close	Demo completed work, review acceptance criteria, document results, and record retrospective actions.

6. Day 1 — Baseline Implementation

Implemented endpoint: GET /api/appointments/summary

Purpose: Return appointment summary information including appointment ID/date/type/status plus the associated patient and doctor names.

6.1 Service Implementation

The current AppointmentService implementation loads appointments with AsNoTracking() and ToListAsync(), then iterates through the appointments. For each appointment, it separately queries Patients and Doctors using FirstOrDefaultAsync() before constructing AppointmentSummaryDto objects.

Important Day 1 finding

The implementation is a useful baseline for demonstrating the N+1 query problem: one query retrieves the appointments, then additional patient/doctor queries are executed repeatedly for the returned appointments.

```

7 namespace CardiacPatientMonitoringSystem.Services;
9 public class AppointmentService : IAppointmentService

203 public async Task<IEnumerable<AppointmentSummaryDto>> GetSummaryAsync()
204 {
205     var appointments = await _context.Appointments
206         .AsNoTracking()
207         .ToListAsync();
208
209     var result = new List<AppointmentSummaryDto>();
210
211     foreach (var appointment in appointments)
212     {
213         var patient = await _context.Patients
214             .AsNoTracking()
215             .FirstOrDefaultAsync(p =>
216                 p.PatientId == appointment.PatientId);
217
218         var doctor = await _context.Doctors
219             .AsNoTracking()
220             .FirstOrDefaultAsync(d =>
221                 d.DoctorId == appointment.DoctorId);
222
223         result.Add(new AppointmentSummaryDto
224         {
225             AppointmentId = appointment.AppointmentId,
226             AppointmentDate = appointment.AppointmentDate,
227             AppointmentType = appointment.AppointmentType,
228             Status = appointment.Status,
229             PatientName = patient is null
230                 ? "Unknown"
231                 : $"{patient.FirstName} {patient.LastName}",
232             DoctorName = doctor is null
233                 ? "Unknown"
234                 : $"{doctor.FirstName} {doctor.LastName}"
235         });
236     }
237
238     return result;
239 }

```

Figure 1 — *AppointmentService.GetSummaryAsync()* implementation.

7. Day 1 — Runtime & Database Evidence

7.1 Application Startup

- The project builds successfully under net10.0.
- The API starts successfully and listens on http://localhost:5180.
- ASP.NET Core reports the Development environment.
- The existing EF Core sensitive-data-logging warning is present in Development mode.
- A Microsoft.OpenApi 2.0.0 package vulnerability warning (NU1903) is reported during restore/build and should be addressed separately.

```

info: Microsoft.EntityFrameworkCore.Database.Command[20101]
      Executed DbCommand (6ms) [Parameters=[], CommandType='Text', CommandTimeout='30']
      SELECT a."AppointmentId", a."AppointmentDate", a."AppointmentType", a."DoctorId", a."Notes", a."PatientId", a."Reason", a."Statu
s"
      FROM "Appointments" AS a
info: 9/6/2026 18:44:53.839 RelationalEventId.CommandExecuted[20101] (Microsoft.EntityFrameworkCore.Database.Command)
      Executed DbCommand (6ms) [Parameters=[], CommandType='Text', CommandTimeout='30']
      SELECT a."AppointmentId", a."AppointmentDate", a."AppointmentType", a."DoctorId", a."Notes", a."PatientId", a."Reason", a."Statu
s"
      FROM "Appointments" AS a
info: Microsoft.EntityFrameworkCore.Database.Command[20101]
      Executed DbCommand (6ms) [Parameters=[@appointment_PatientId='4'], CommandType='Text', CommandTimeout='30']
      SELECT p."PatientId", p."Address", p."DateOfBirth", p."Email", p."EmergencyContactName", p."EmergencyContactPhone", p."FirstName
", p."Gender", p."LastName", p."MedicalNotes", p."MedicalRecordNumber", p."Phone", p."UserId"
      FROM "Patients" AS p
      WHERE p."PatientId" = @appointment_PatientId
      LIMIT 1
info: 9/6/2026 18:44:53.873 RelationalEventId.CommandExecuted[20101] (Microsoft.EntityFrameworkCore.Database.Command)
      Executed DbCommand (6ms) [Parameters=[@appointment_PatientId='4'], CommandType='Text', CommandTimeout='30']
      SELECT p."PatientId", p."Address", p."DateOfBirth", p."Email", p."EmergencyContactName", p."EmergencyContactPhone", p."FirstName
", p."Gender", p."LastName", p."MedicalNotes", p."MedicalRecordNumber", p."Phone", p."UserId"
      FROM "Patients" AS p
      WHERE p."PatientId" = @appointment_PatientId
      LIMIT 1
info: Microsoft.EntityFrameworkCore.Database.Command[20101]
      Executed DbCommand (5ms) [Parameters=[@appointment_DoctorId='4'], CommandType='Text', CommandTimeout='30']
      SELECT d."DoctorId", d."Email", d."FirstName", d."LastName", d."LicenseNumber", d."Phone", d."Specialization", d."UserId"
      FROM "Doctors" AS d
      WHERE d."DoctorId" = @appointment_DoctorId
      LIMIT 1
info: 9/6/2026 18:44:53.899 RelationalEventId.CommandExecuted[20101] (Microsoft.EntityFrameworkCore.Database.Command)
      Executed DbCommand (5ms) [Parameters=[@appointment_DoctorId='4'], CommandType='Text', CommandTimeout='30']
      SELECT d."DoctorId", d."Email", d."FirstName", d."LastName", d."LicenseNumber", d."Phone", d."Specialization", d."UserId"
      FROM "Doctors" AS d
      WHERE d."DoctorId" = @appointment_DoctorId
  
```

Figure 2 — EF Core SQL activity observed while executing the summary request.

7.2 What the SQL Log Shows

- A SELECT is executed against the "Appointments" table.
- The application then issues SELECT statements against "Patients" using the appointment PatientId.
- It also issues SELECT statements against "Doctors" using the appointment DoctorId.
- The repeated patient/doctor SELECT pattern is the concrete Day 1 evidence that motivates the N+1 optimization work planned for the next step.

8. Day 1 — Postman Verification

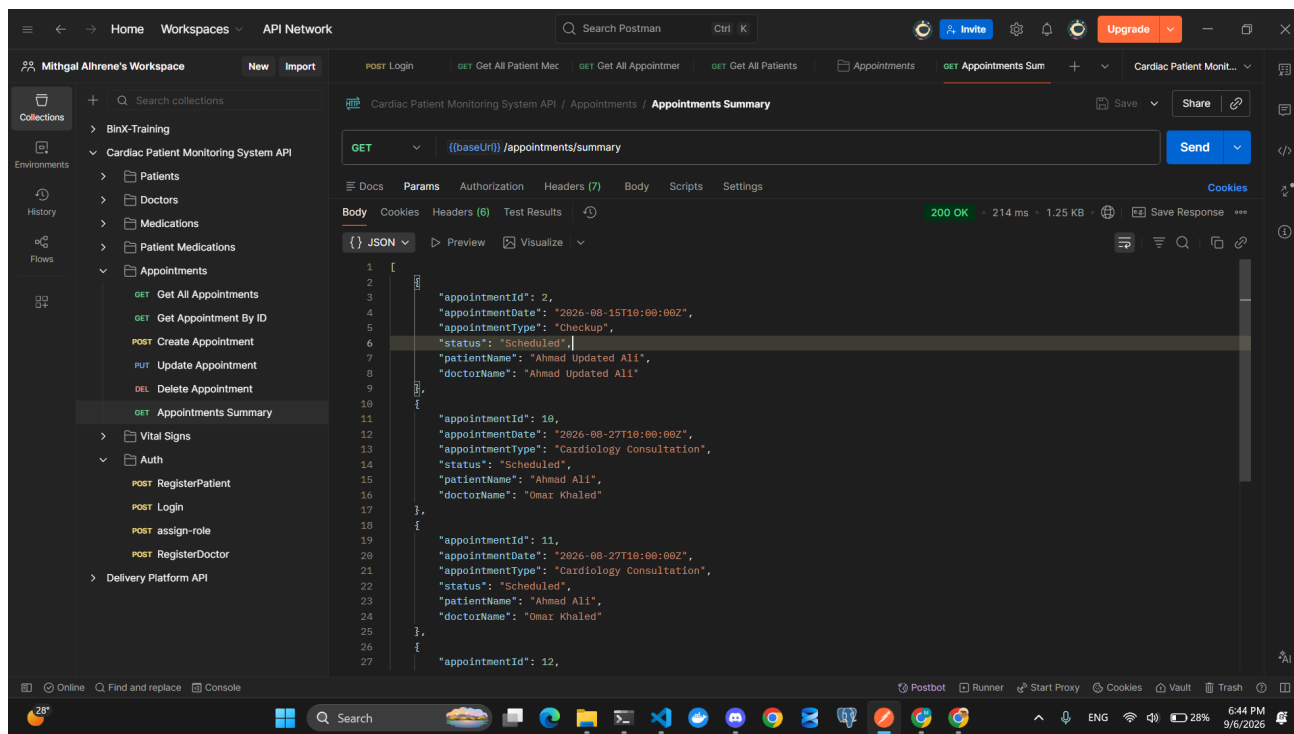


Figure 3 — Postman verification of GET /api/appointments/summary.

Verification item	Observed result
Request	GET {{baseUrl}}/appointments/summary
HTTP status	200 OK
Response type	JSON array
Summary fields	appointmentId, appointmentDate, appointmentType, status, patientName, doctorName
Example data	Appointment 2, 10, 11 and additional returned records are visible in the response
Request timing shown	214 ms in the captured Postman response

9. N+1 Query Analysis

Current pattern: The service first retrieves all appointments. Inside the foreach loop, it then performs one patient query and one doctor query per appointment.

Stage	Current behavior	Performance implication
1	Load appointments	1 database query
2	Load patient for each appointment	Repeated database queries
3	Load doctor for each appointment	Repeated database queries
Result	Total query count grows with the number of appointments	Classic N+1-style access pattern

Optimization direction

The next implementation step should move the related-data retrieval into a more efficient EF Core/LINQ query so the database can return the required summary data with substantially fewer round trips.

10. Acceptance Criteria

- GET /api/appointments/summary returns the expected appointment summary data.
- The endpoint returns an appropriate successful HTTP response for valid requests.
- The response contains appointment, patient, and doctor information required by the summary DTO.
- The optimized implementation reduces unnecessary database round trips compared with the Day 1 baseline.
- The optimization preserves existing API behavior and does not introduce unhandled exceptions.
- The work is committed using the established Git feature-branch workflow and clear commit messages.
- The completed endpoint/feature is ready for mentor review according to the sprint workflow.

11. Definition of Done

For sprint work, completion follows the program's professional sprint criteria: the endpoint must satisfy its acceptance criteria, be committed to the correct feature branch with descriptive messages, have the required review flow, and meet the project's testing and quality expectations.

Check	Day 1 state
Endpoint implemented	Yes
Endpoint verified in Postman	Yes — 200 OK
Database behavior inspected	Yes
N+1 pattern identified	Yes
Optimization applied	Next task
Redis caching applied	Later sprint task
Performance comparison completed	Pending optimization

12. Day 1 Deliverables

- Appointments Summary endpoint implementation.
- AppointmentSummaryDto response structure.
- Runtime/database evidence showing the generated EF Core queries.
- Postman verification showing a 200 OK response.
- Identification of the N+1 query pattern to guide the optimization work.

13. Sprint Review & Retrospective — To Be Completed

Area	Record
What went well	To be completed during Sprint Review
What needs improvement	To be completed after optimization/performance work
Performance evidence	To be recorded after the optimized implementation
Next action	Optimize the summary query and validate the reduction in database round trips

14. Source & Evidence Notes

This document combines the official BinX Tech internship sprint structure with the Day 1 implementation evidence supplied for the Cardiac Patient Monitoring System. The Week 8 scope is aligned with the program outline: Sprint 3 covers advanced queries (LINQ), Redis caching, and performance tuning. The screenshots and terminal output document the actual Day 1 implementation and verification.

BINX TECH • SPRINT 3 • DAY 1